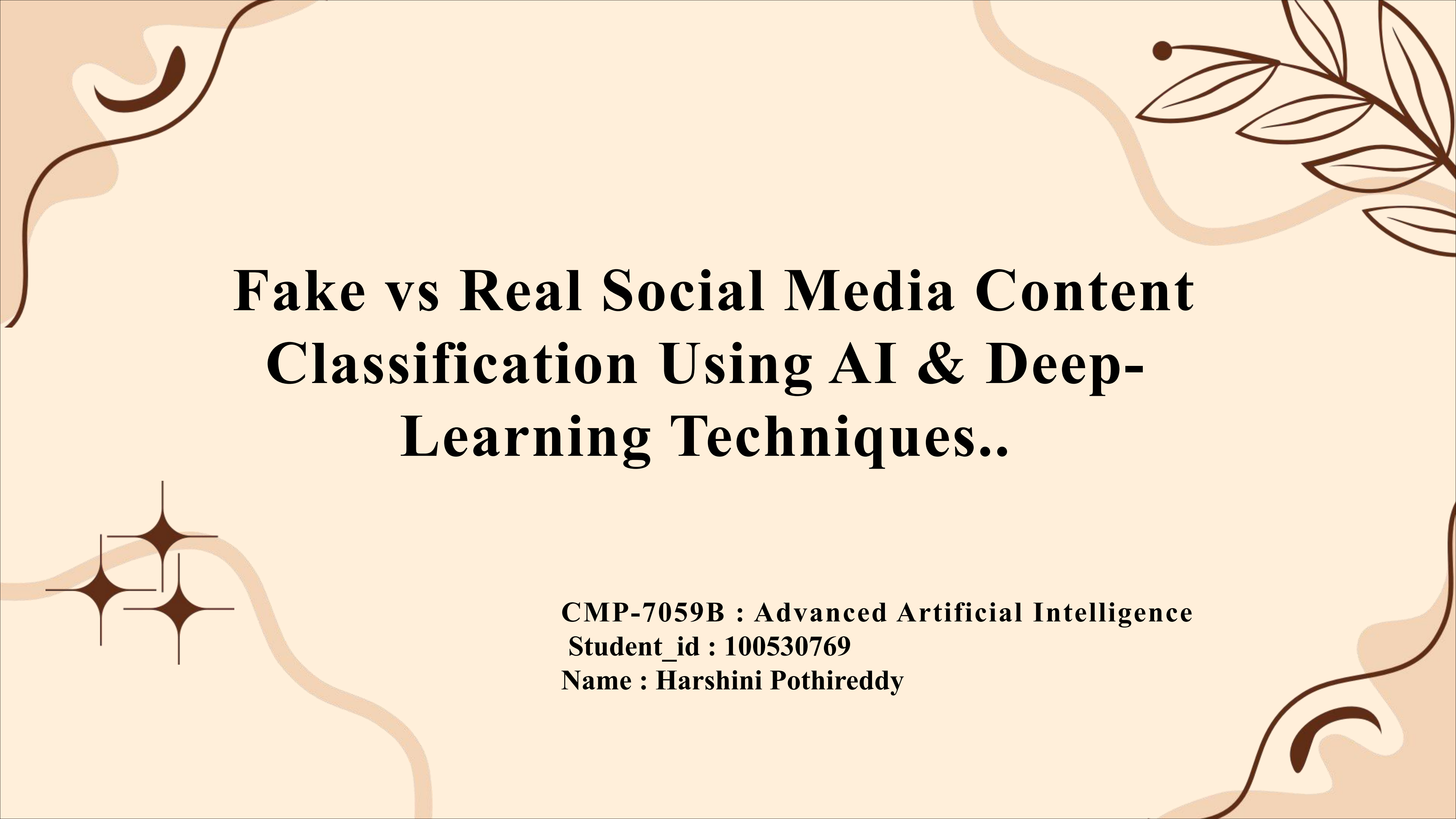




Fake vs Real Social Media Content Classification Using AI & Deep- Learning Techniques..

CMP-7059B : Advanced Artificial Intelligence

Student_id : 100530769

Name : Harshini Pothireddy

PROJECT OVERVIEW

- This project focuses on automatically classifying social media posts as Real or Fake using Natural Language Processing and neural networks.
- The dataset contains over 89,000 posts, each linked to a news headline and labelled through a majority - vote process.
- I implemented two different models :
 - a Multi-Layer Perceptron (MLP) using TF-IDF features
 - Convolutional Neural Network (CNN) using tokenised sequences
- I also used topic modelling (LDA) using classical NLP techniques to uncover the main themes in the dataset.
- All models were trained, saved, and used for real-time predictions during the demonstration.

PROJECT WORKFLOW

- The workflow begins with loading and understanding the dataset, including headlines, posts, and labels.
- I applied text preprocessing to clean the data and make it suitable for modelling.
- The cleaned text was converted into two types of numerical representations:
 1. TF-IDF vectors for the MLP
 2. Tokenised and padded sequences for the CNN
- I trained both models, tuned their hyperparameters, and evaluated their performance.
- Finally, I used LDA topic modelling to discover the main themes discussed in the posts.

DATA & PRE-PROCESSING

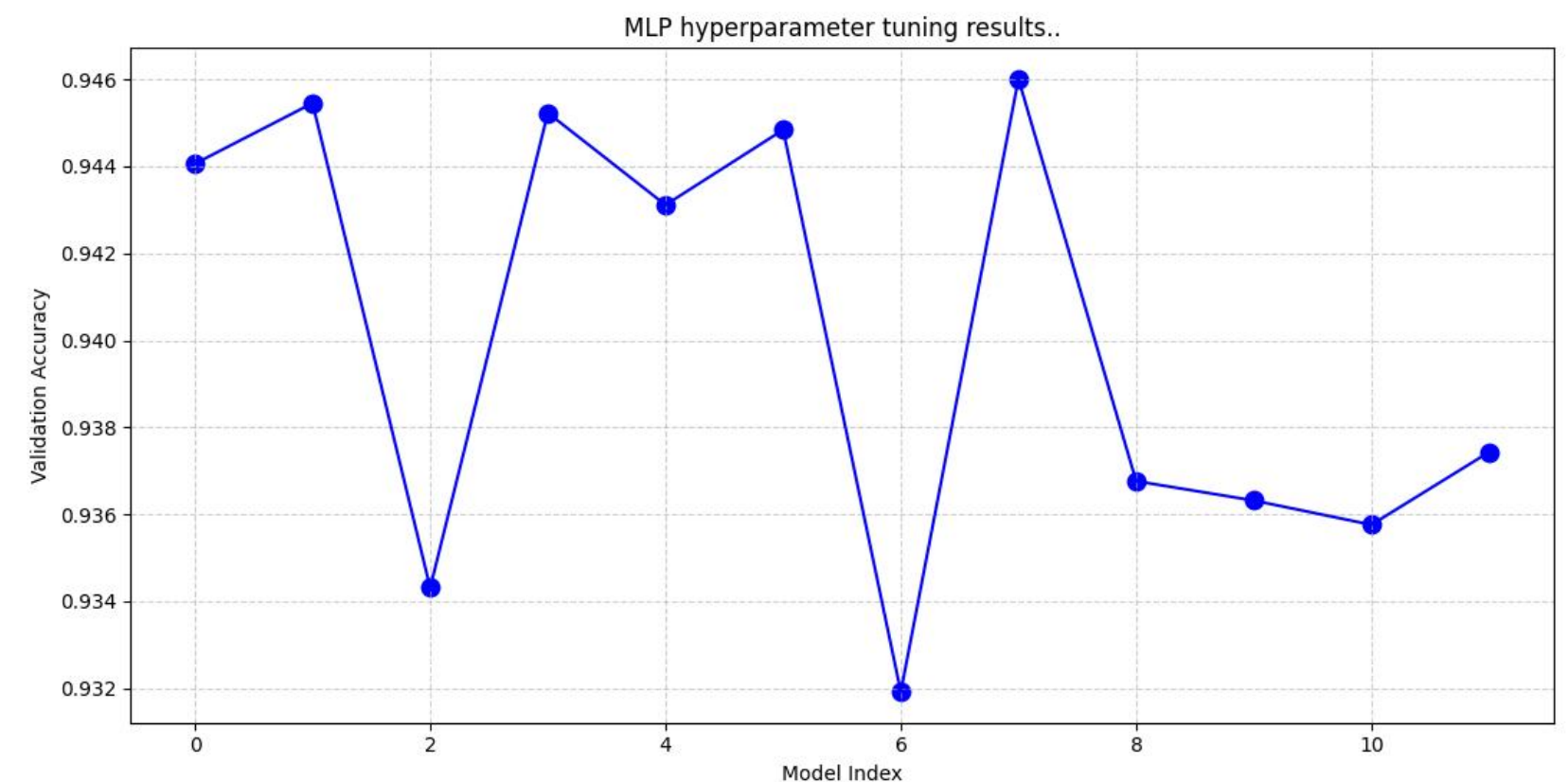
- Each data point includes a headline, the truthfulness of that headline, the post text, and the final real/fake label.
- I cleaned the text using several NLP pre-processing algorithms like:
 1. **lowercasing** - to remove case sensitivity
 2. **punctuation removal** - to reduce Noise
 3. **stopword removal** - to remove common but uninformative word
 4. **stemming** - to reduce words to their root form
- I combined the headline and post into a single document so the models could learn from full context.
- After cleaning, I split the dataset into training, validation, and test sets to ensure fair evaluation.
- This preprocessing step ensures that both the MLP and CNN receive consistent, high-quality input.

MLP Model (TF-IDF & Hyperparameter Tuning)

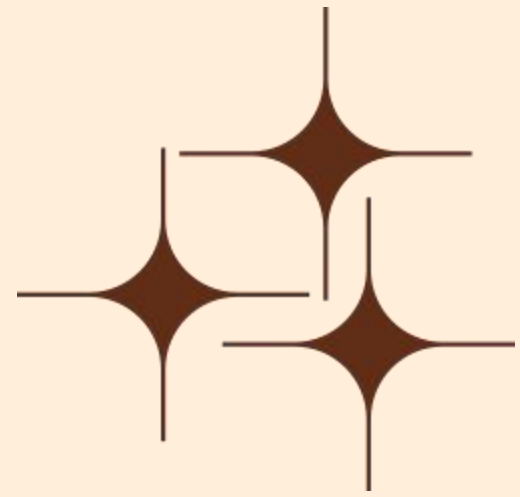
- The MLP uses TF-IDF, which measures how important each word is in a document compared to the whole dataset.
- TF-IDF creates a numerical feature vector for each document, which the MLP can learn from.
- I experimented with different hidden-layer sizes, activation functions, and learning rates.
- Hyperparameter tuning helped me identify the best-performing MLP configuration.

Graph explanation:

- This graph shows how validation accuracy changes across different MLP configurations.
- Each point represents a different combination of hyperparameters.
- Accuracy ranges from 0.932 to 0.946, showing that tuning has a meaningful impact.
- This helped me select the best MLP model for the final evaluation.



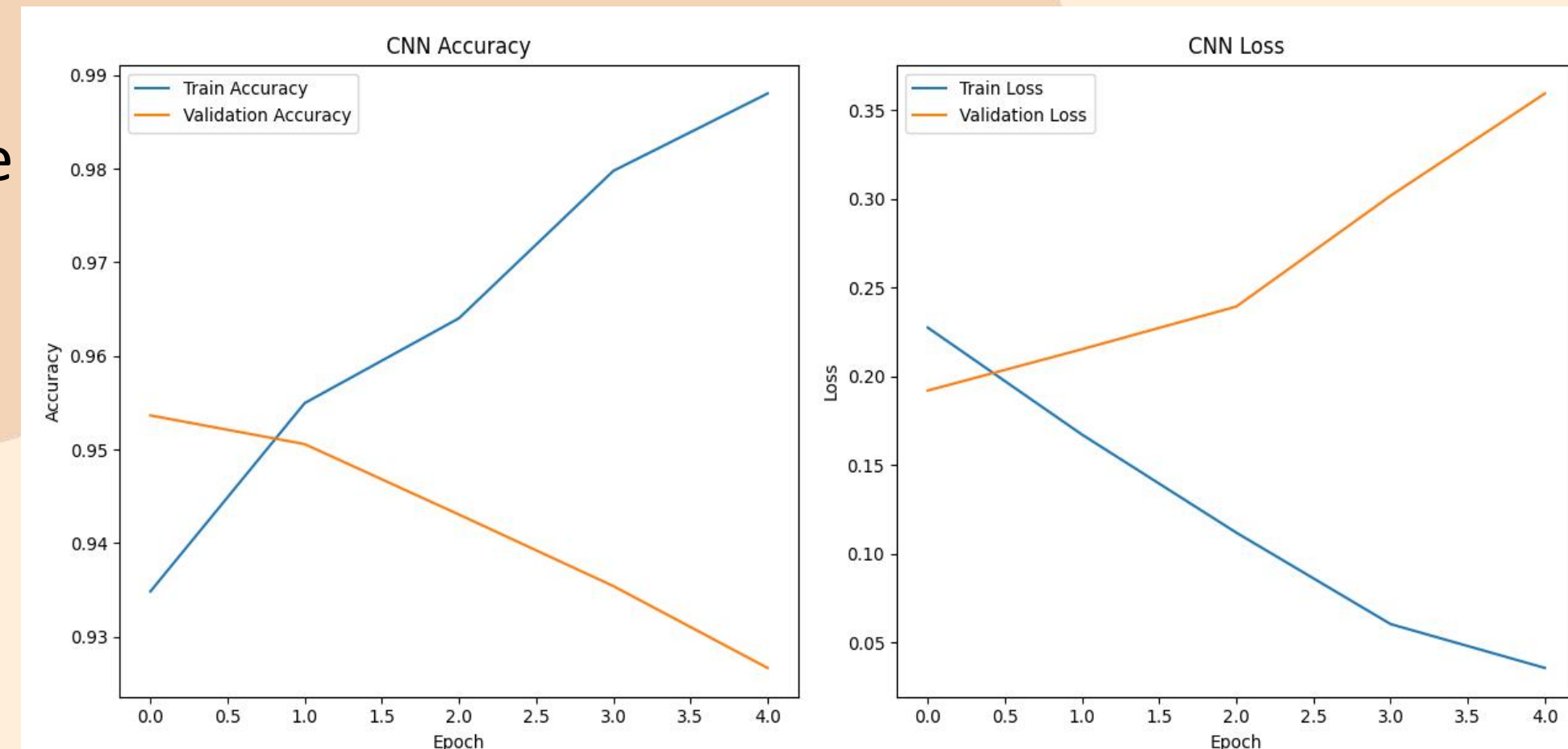
CNN Model (Tokenisation + Deep Learning)



- The CNN works on tokenised and padded sequences, which preserve word order and context.
- The model includes:
 - an Embedding layer to learn word representations
 - a Conv1D layer to detect patterns like phrases
 - a GlobalMaxPooling layer to extract the strongest features
 - dense layers for final classification
- I tuned hyperparameters such as embedding size, number of filters, and kernel size.

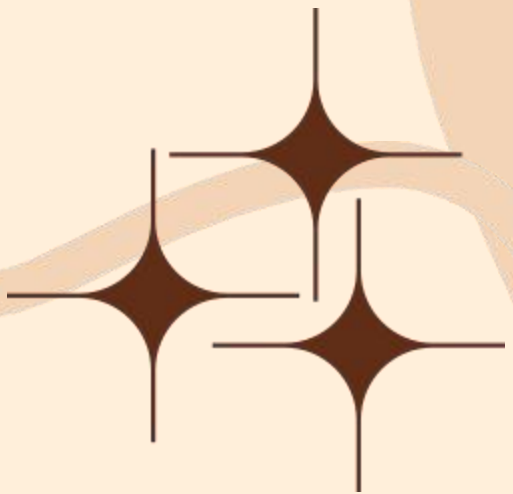
Graph explanation:

- Training accuracy increases to almost 99%, while validation accuracy decreases slightly.
- Training loss drops sharply, but validation loss increases.
- This pattern indicates overfitting, but the CNN still outperformed the MLP on the test set.



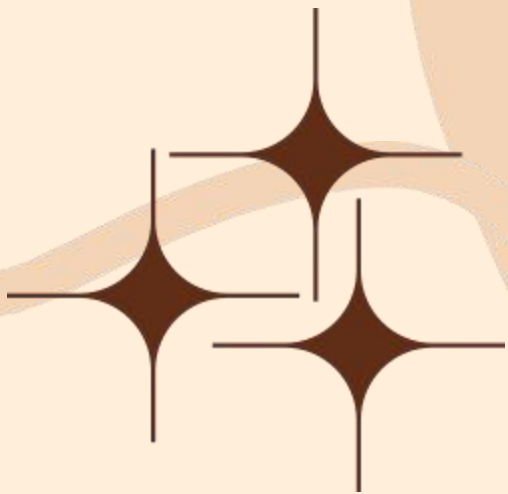
Results & Evaluation

- I evaluated both models using accuracy, loss curves, and confusion matrices.
- The CNN achieved higher accuracy because it captures context and word-order patterns.
- The MLP still performed well and provided a strong classical baseline.
- I tested the saved models on new text inputs to confirm they were robust and ready for real-time use.
- These evaluations helped me understand how different algorithms behave on the same dataset and which features contribute most to performance.



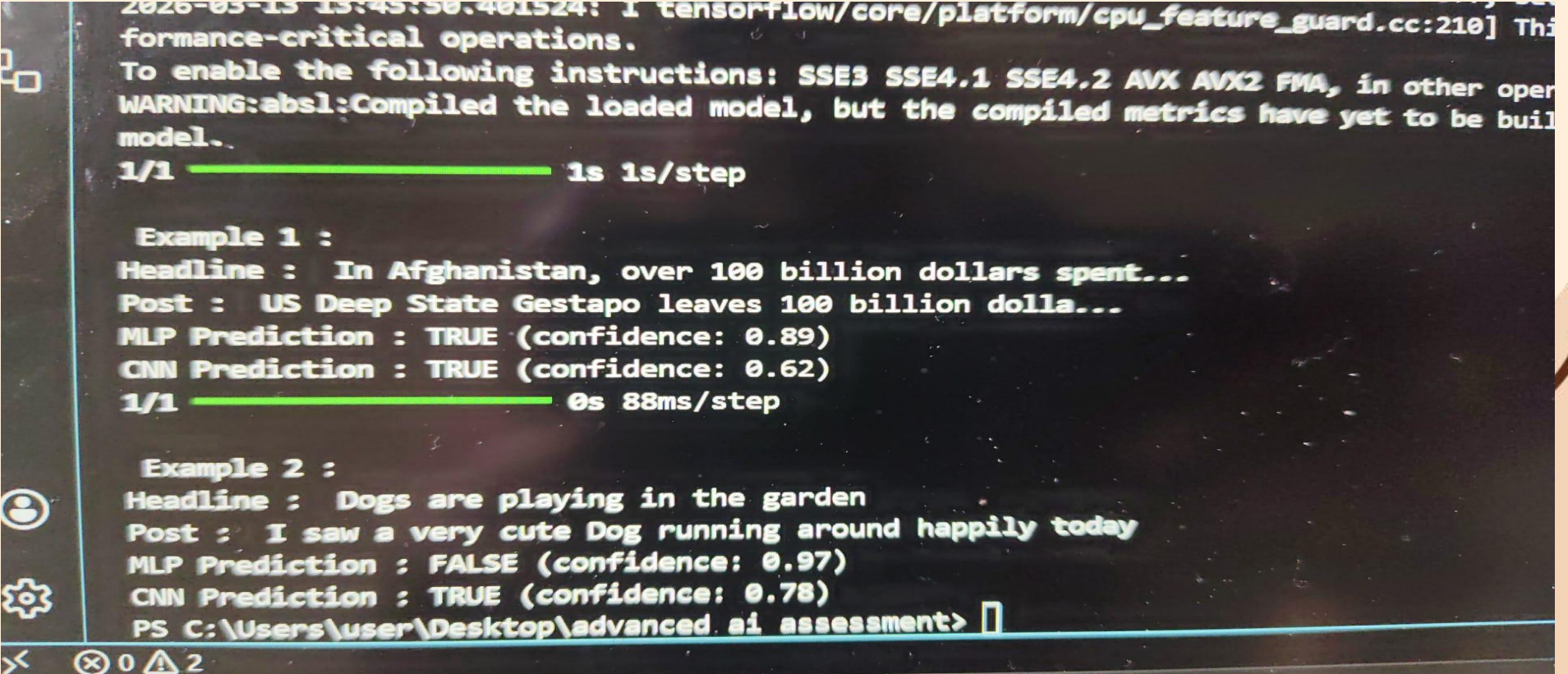
Topic Discovery (LDA)

- I used Latent Dirichlet Allocation (LDA) to discover hidden themes in the dataset.
- LDA assumes each document is a mixture of topics and each topic is a mixture of words.
- After cleaning the text, I experimented with both Bag-of-Words(BoW) and TF-IDF representations.
- LDA revealed clear topics such as politics, public health, economy, and social issues.
- This helped me understand the dataset beyond classification and provided deeper insights into the discussions happening in the posts.
- I saved the best LDA model to analyse new text during the demonstration.



OUTPUT

- Each example takes a headline + social media post as input and checks if they align.
- Both models (MLP using TF-IDF, and CNN using embeddings) output a TRUE/FALSE label with a confidence score.
- **Example 1:** Both models predict TRUE, meaning the post matches the headline.
- **Example 2:** MLP predicts FALSE while CNN predicts TRUE, showing how different models interpret text differently



```
2026-03-13 13:45:50.401524: I tensorflow/core/platform/cpu_feature_guard.cc:210] This binary is not configured to run on this hardware. This may cause a performance-critical operation.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, you need to compile TensorFlow with the appropriate flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built.
1/1 ----- 1s 1s/step

Example 1 :
Headline : In Afghanistan, over 100 billion dollars spent...
Post : US Deep State Gestapo leaves 100 billion dolla...
MLP Prediction : TRUE (confidence: 0.89)
CNN Prediction : TRUE (confidence: 0.62)
1/1 ----- 0s 88ms/step

Example 2 :
Headline : Dogs are playing in the garden
Post : I saw a very cute Dog running around happily today
MLP Prediction : FALSE (confidence: 0.97)
CNN Prediction : TRUE (confidence: 0.78)
PS C:\Users\user\Desktop\advanced.ai.assessment>
```